

Otimização Bioobjetivo da Alocação de Máquinas Virtuais em Computação em Nuvem com Colônia de Formigas (VMPACS)

Seminário de Algoritmos Bioinspirados

Lucas Rivetti Ian Nunes

Junho de 2026

Resumo

Este trabalho estuda e aplica um algoritmo bioinspirado *multiobjetivo* — o **VMPACS** (*Virtual Machine Placement Ant Colony System*), proposto por Gao et al. (2013) — ao problema biobjetivo de **alocação de máquinas virtuais (VMs)** em *data centers* de nuvem. O objetivo é encontrar um conjunto de soluções não-dominadas (fronteira de Pareto) que minimizem *simultaneamente* o **consumo de energia** e o **desperdício de recursos**. Implementamos o VMPACS fielmente ao artigo e o comparamos com o **NSGA-II**, um algoritmo genético multiobjetivo clássico. Os experimentos seguem a metodologia do artigo (instâncias com diferentes correlações entre CPU e memória) e a qualidade das soluções é avaliada por meio de **quatro métricas** de desempenho (Hipervolume, IGD, ONVG e *Spacing*), além dos indicadores práticos de energia, desperdício e número de servidores. Os resultados mostram que ambos os algoritmos obtêm empacotamentos compactos e de boa qualidade; o NSGA-II converge, em geral, para soluções de menor energia e desperdício, enquanto o VMPACS tende a fornecer fronteiras ligeiramente mais diversificadas.

Sumário

1	Descrição do problema	1
2	Objetivos de otimização	2
3	Formulação matemática	2
4	Algoritmos utilizados	3
4.1	VMPACS — Sistema de Colônia de Formigas	3
4.2	NSGA-II — Algoritmo Genético Multiobjetivo (comparação)	4
5	Metodologia experimental	4
6	Resultados	4
7	Análise dos resultados	6
8	Conclusões	8

1 Descrição do problema

A virtualização é a tecnologia central da computação em nuvem: ela permite que vários ambientes isolados (as *máquinas virtuais*, VMs) compartilhem um mesmo servidor físico. A **alocação de máquinas virtuais** (*VM placement*) é o processo de decidir *em qual servidor físico cada VM será executada*. Essa decisão é crítica porque governa diretamente o **custo operacional** de um *data center*, dominado pelo consumo de energia, e o **aproveitamento dos recursos** (CPU e memória).

Do ponto de vista do provedor, deseja-se *consolidar* as VMs no menor número possível de servidores (desligando os ociosos para poupar energia), mas sem sobrecarregar nem desbalancear os servidores ativos. O problema é uma variante do *empacotamento vetorial* (*vector bin packing*), que é NP-difícil: para n VMs e m servidores existem m^n alocações possíveis, tornando a enumeração completa inviável. Por isso recorremos a **meta-heurísticas bioinspiradas**.

Por que é um problema multiobjetivo. Reduzir o número de servidores diminui a energia, mas pode forçar empacotamentos desbalanceados (por exemplo, um servidor com a CPU cheia e a memória quase vazia), elevando o desperdício. Tratamos, portanto, energia e desperdício como *dois objetivos conflitantes* e buscamos o conjunto de soluções de melhor compromisso (a fronteira de Pareto), em vez de uma única solução.

2 Objetivos de otimização

Cada servidor é caracterizado por duas dimensões de recurso: **CPU** e **memória**. Sejam U_j^p e U_j^m as utilizações (frações) de CPU e memória do servidor j .

- **Objetivo 1 — Consumo de energia.** Medições mostram que a potência de um servidor cresce linearmente com a utilização de CPU. Servidores ociosos são desligados. A potência do servidor j é

$$P_j = (P_j^{busy} - P_j^{idle}) U_j^p + P_j^{idle}, \quad \text{se } U_j^p > 0, \text{ senão } 0, \quad (1)$$

com $P^{idle} = 162$ W e $P^{busy} = 215$ W (valores do artigo). Minimizar $\sum_j P_j$ equivale, na prática, a *minimizar o número de servidores ligados*.

- **Objetivo 2 — Desperdício de recursos.** Para aproveitar bem *as duas* dimensões e mantê-las equilibradas, o desperdício do servidor j é

$$W_j = \frac{|L_j^p - L_j^m| + \varepsilon}{U_j^p + U_j^m}, \quad (2)$$

onde L_j^p e L_j^m são os recursos de CPU e memória *restantes* no servidor e $\varepsilon = 10^{-4}$. O termo $|L_j^p - L_j^m|$ penaliza o *desbalanceamento* entre as sobras de CPU e memória.

3 Formulação matemática

Sejam n VMs ($i \in I$) a serem alocadas em m servidores ($j \in J$). Cada VM i tem demanda de CPU R_i^p e de memória R_i^m ; cada servidor j tem limites (*thresholds*) T_j^p e T_j^m . Definimos as variáveis binárias:

$$x_{ij} = \begin{cases} 1, & \text{se a VM } i \text{ é alocada no servidor } j \\ 0, & \text{caso contrário} \end{cases} \quad y_j = \begin{cases} 1, & \text{se o servidor } j \text{ está em uso} \\ 0, & \text{caso contrário.} \end{cases}$$

O problema biobjetivo é:

$$\min f_1 = \sum_{j=1}^m y_j \left[(P_j^{busy} - P_j^{idle}) \sum_{i=1}^n x_{ij} R_i^p + P_j^{idle} \right] \quad (3)$$

$$\min f_2 = \sum_{j=1}^m y_j \frac{|(T_j^p - \sum_i x_{ij} R_i^p) - (T_j^m - \sum_i x_{ij} R_i^m)| + \varepsilon}{\sum_i x_{ij} R_i^p + \sum_i x_{ij} R_i^m} \quad (4)$$

sujeito a:

$$\sum_{j=1}^m x_{ij} = 1, \quad \forall i \in I, \quad (5)$$

$$\sum_{i=1}^n R_i^p x_{ij} \leq T_j^p y_j, \quad \forall j \in J, \quad (6)$$

$$\sum_{i=1}^n R_i^m x_{ij} \leq T_j^m y_j, \quad \forall j \in J, \quad (7)$$

$$x_{ij}, y_j \in \{0, 1\}, \quad \forall i \in I, \forall j \in J. \quad (8)$$

A restrição (5) garante que cada VM seja alocada em exatamente um servidor; (6) e (7) impõem os limites de CPU e memória de cada servidor; (8) define o domínio das variáveis. Usamos $T_j^p = T_j^m = 0,9$ (90%) para evitar a degradação de desempenho que ocorre próximo de 100% de utilização.

4 Algoritmos utilizados

4.1 VMPACS — Sistema de Colônia de Formigas

O VMPACS é uma adaptação do *Ant Colony System* (ACS) ao problema. A metáfora biológica é a de formigas que, ao buscar alimento, depositam **feromônio** nos caminhos percorridos; caminhos bons acumulam mais feromônio e atraem mais formigas. Aqui, cada “formiga” constrói uma alocação completa e os bons *movimentos* (alocar a VM i no servidor j) recebem mais feromônio.

Feromônio e heurística. O feromônio τ_{ij} indica a conveniência de colocar a VM i no j -ésimo servidor aberto. É inicializado com $\tau_0 = 1/[n(P'(S_0) + W(S_0))]$, onde S_0 é a solução da heurística *First-Fit Decreasing* (FFD). A informação heurística combina a contribuição parcial de cada movimento aos dois objetivos:

$$\eta_{ij} = \eta_{ij}^{(1)} + \eta_{ij}^{(2)}, \quad \eta_{ij}^{(1)} = \frac{1}{\varepsilon + \sum_{v=1}^j P_v / P_v^{\max}}, \quad \eta_{ij}^{(2)} = \frac{1}{\varepsilon + \sum_{v=1}^j W_v}. \quad (9)$$

Construção da solução (regra pseudoaleatória proporcional). Para escolher a próxima VM a empacotar no servidor atual, sorteia-se $q \sim U[0, 1]$:

$$i = \begin{cases} \arg \max_{u \in \Omega} \{\alpha \tau_{uj} + (1 - \alpha) \eta_{uj}\}, & q \leq q_0 \text{ (intensificação)} \\ s \text{ sorteado por } p_{uj} = \frac{\alpha \tau_{uj} + (1 - \alpha) \eta_{uj}}{\sum_{w \in \Omega} \alpha \tau_{wj} + (1 - \alpha) \eta_{wj}}, & q > q_0 \text{ (exploração)}, \end{cases} \quad (10)$$

onde Ω é o conjunto de VMs que ainda cabem no servidor atual e α pondera feromônio *vs.* heurística.

Atualização do feromônio. Há dois passos. A atualização *local*, aplicada a cada movimento construído, evita estagnação:

$$\tau_{ij} \leftarrow (1 - \rho_l) \tau_{ij} + \rho_l \tau_0. \quad (11)$$

A atualização *global*, aplicada apenas às soluções da fronteira de Pareto (arquivo externo) ao fim de cada iteração, reforça os bons movimentos:

$$\tau_{ij} \leftarrow (1 - \rho_g) \tau_{ij} + \frac{\rho_g \lambda}{P'(S) + W(S)}, \quad \lambda = \frac{N_A}{t - N_{I_s} + 1}, \quad (12)$$

em que t é a iteração atual e N_{I_s} a iteração em que a solução S entrou no arquivo. O Algoritmo 1 resume o procedimento.

Algoritmo 1: VMPACS

Entrada: VMs e servidores com demandas e limites; parâmetros $\alpha, \rho_l, \rho_g, q_0, N_A, M$.

Saída: arquivo de Pareto P .

```

1  Inicializa  $P \leftarrow \emptyset$  e todo  $\tau_{ij} \leftarrow \tau_0$ 
2  para  $t = 1$  até  $M$  faça
3    para cada formiga  $1..N_A$  faça
4      abre um servidor; enquanto houver VMs que cabem: escolhe uma por (10)
5      e aplica a atualização local (11); repete abrindo novos servidores
6      avalia  $(f_1, f_2)$  e atualiza o arquivo de não-dominadas  $P$ 
7    para cada solução  $S \in P$ : aplica a atualização global (12)
8  retorna  $P$ 
```

Figura 1: Pseudocódigo do VMPACS.

4.2 NSGA-II — Algoritmo Genético Multiobjetivo (comparação)

Para comparação, usamos o **NSGA-II** (Deb et al., 2002), o algoritmo genético multiobjetivo mais difundido — assim como o artigo original compara o VMPACS com um algoritmo genético (MGGA). Seus dois mecanismos centrais são: (i) a *ordenação por não-dominância*, que classifica a população em frentes de Pareto sucessivas; e (ii) a *distância de multidão* (*crowding*), que preserva a diversidade ao longo da fronteira. Cada indivíduo é uma **permutação** das VMs, decodificada pela heurística *first-fit* (garantindo soluções sempre válidas); os operadores são o *Order Crossover* (OX) e a mutação por troca (*swap*). A seleção usa torneio binário baseado em (frente, *crowding*) e elitismo ($\mu + \lambda$).

5 Metodologia experimental

Instâncias. Geramos instâncias com $n = 100$ VMs e $m = 100$ servidores homogêneos, reproduzindo o gerador correlacionado do artigo: a demanda de CPU é sorteada em $[0, 2\bar{R}_p)$ e a de memória em $[0, \bar{R}_m)$, com $\bar{R}_p = \bar{R}_m = 0,25$; um parâmetro $P \in [0, 1]$ controla a **correlação** entre CPU e memória. Usamos cinco níveis, $P \in \{0; 0,25; 0,5; 0,75; 1,0\}$, correspondentes a coeficientes de correlação aproximados de $-0,75$ (negativa forte) a $+0,75$ (positiva forte).

Parâmetros dos algoritmos. VMPACS (valores do artigo): $N_A = 10$ formigas, $M = 100$ iterações, $\alpha = 0,45$, $\rho_l = \rho_g = 0,35$, $q_0 = 0,8$. NSGA-II: população 50, 20 gerações, taxa de mutação 0,4. Os orçamentos foram *equiparados* em ≈ 1000 avaliações de solução por execução, para uma comparação justa.

Métricas de desempenho. Avaliamos a qualidade das fronteiras com quatro métricas consagradas:

- **Hipervolume (HV)** — área do espaço de objetivos dominada pela fronteira em relação a um ponto de referência; *maior é melhor* (mede convergência e diversidade).
- **IGD** (*Inverted Generational Distance*) — distância média de uma fronteira de referência até a fronteira obtida; *menor é melhor*.
- **ONVG** — número de soluções não-dominadas encontradas; cardinalidade da fronteira.
- **Spacing (SP)** — uniformidade do espaçamento entre as soluções; *menor é melhor*.

Para HV e IGD, os objetivos são normalizados em $[0, 1]$ por nível de correlação, e a fronteira de referência do IGD é a união das soluções não-dominadas de ambos os algoritmos. Cada configuração é executada $R = 10$ vezes de forma independente; reportamos média $\pm$ desvio-padrão. Para visualizar o trade-off real, calculamos ainda uma fronteira de Pareto de *referência* (varredura do número de servidores com busca local, módulo `referencia.py`).

6 Resultados

A Figura 2 compara os **indicadores práticos** (energia, desperdício e número de servidores) entre os algoritmos, nos cinco níveis de correlação. A Figura 3 apresenta as **métricas multiobjetivo** (Hipervolume e IGD) e exemplos de **fronteiras de Pareto** no espaço de objetivos. As Tabelas 1, 2 e 3 trazem os valores numéricos.

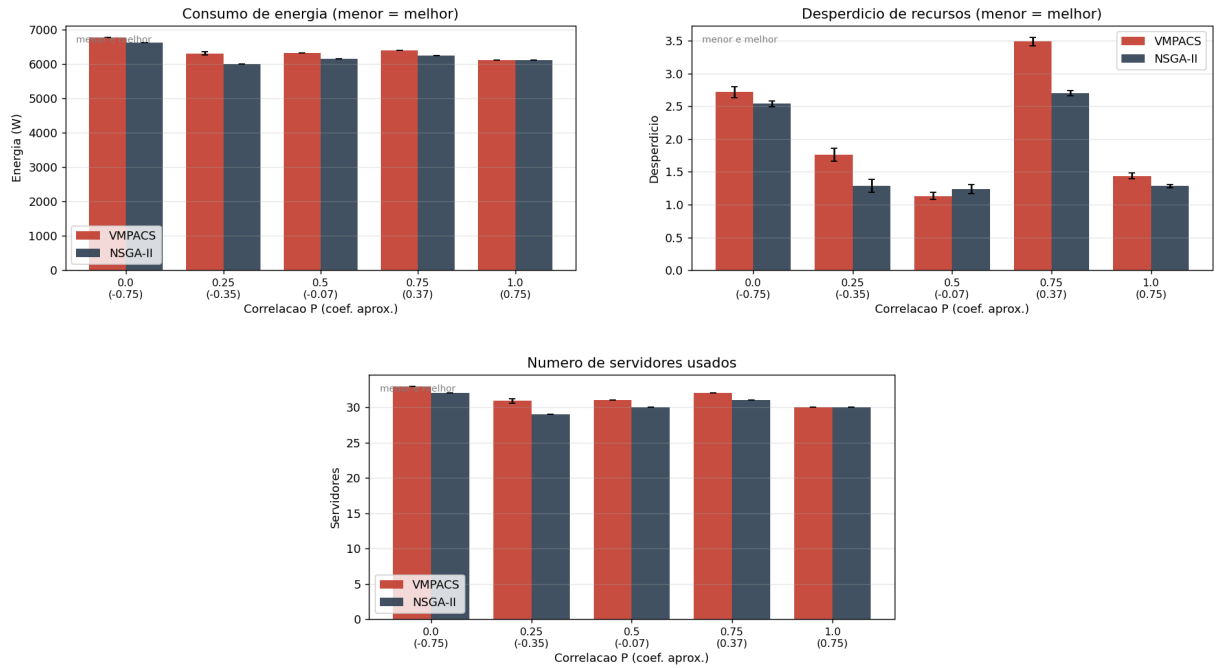


Figura 2: Indicadores práticos por nível de correlação P (coeficiente aproximado entre parênteses): energia, desperdício e número de servidores. Em todos, *menor é melhor*.

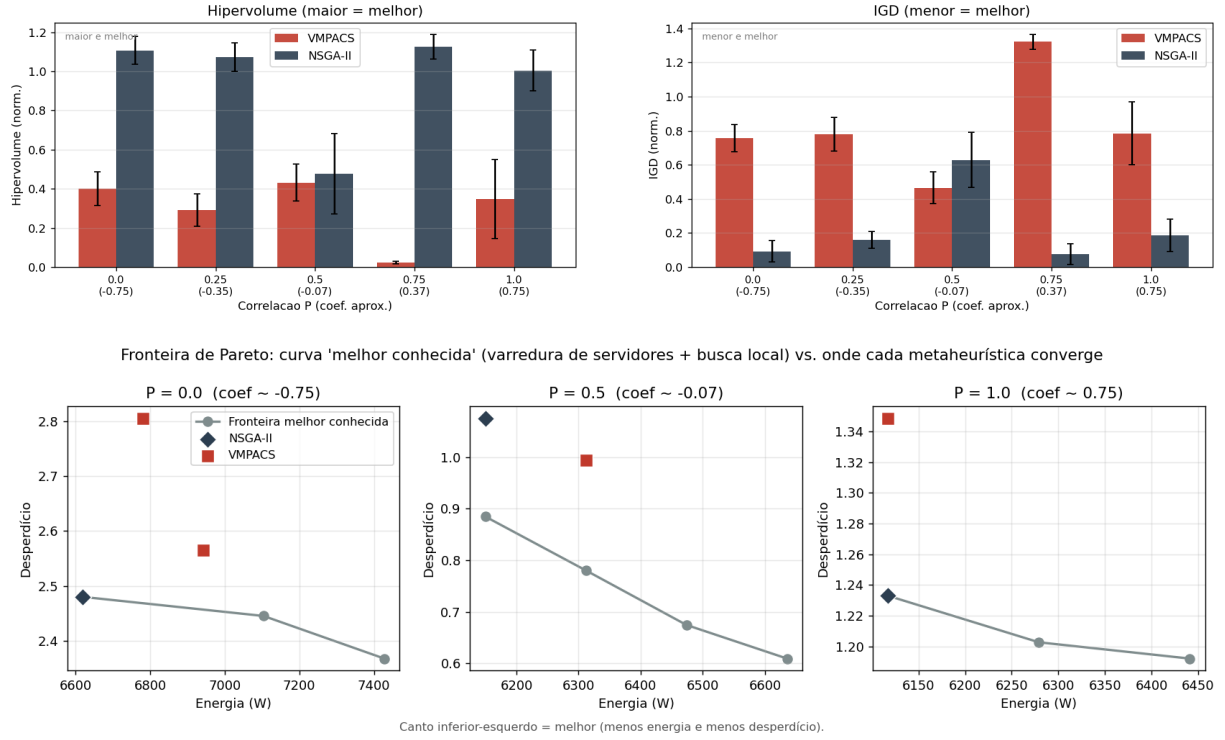


Figura 3: Métricas multiobjetivo (Hypervolume — maior é melhor; IGD — menor é melhor) e fronteiras de Pareto. Na fronteira, a curva cinza é a *melhor conhecida* (varredura do número de servidores + busca local) e os marcadores mostram onde cada metaheurística converge. O canto inferior-esquerdo corresponde às melhores soluções.

Tabela 1: Energia, desperdício e número de servidores (média $\pm$ desvio).

P	coef.	Algoritmo	Energia (W)	Desperdício	Servidores
0,00	-0,75	VMPACS	6780,2 $\pm$ 0,0	2,720 $\pm$ 0,084	33,0
		NSGA-II	6618,2 $\pm$ 0,0	2,540 $\pm$ 0,041	32,0
0,25	-0,35	VMPACS	6302,4 $\pm$ 48,6	1,761 $\pm$ 0,097	30,9
		NSGA-II	5994,6 $\pm$ 0,0	1,286 $\pm$ 0,098	29,0
0,50	-0,07	VMPACS	6311,8 $\pm$ 0,0	1,131 $\pm$ 0,054	31,0
		NSGA-II	6149,8 $\pm$ 0,0	1,234 $\pm$ 0,069	30,0
0,75	+0,37	VMPACS	6402,6 $\pm$ 0,0	3,492 $\pm$ 0,066	32,0
		NSGA-II	6240,6 $\pm$ 0,0	2,700 $\pm$ 0,042	31,0
1,00	+0,75	VMPACS	6117,2 $\pm$ 0,0	1,440 $\pm$ 0,048	30,0
		NSGA-II	6117,2 $\pm$ 0,0	1,282 $\pm$ 0,025	30,0

Tabela 2: ONVG e *Spacing* (média).

P	Algoritmo	ONVG	SP
0,00	VMPACS	2,1	0,018
	NSGA-II	1,1	0,000
0,25	VMPACS	2,9	0,097
	NSGA-II	1,4	0,000
0,50	VMPACS	1,2	0,000
	NSGA-II	1,2	0,000
0,75	VMPACS	1,0	0,000
	NSGA-II	1,3	0,000
1,00	VMPACS	1,0	0,000
	NSGA-II	1,0	0,000

Tabela 3: Hipervolume e IGD (média).

P	Algoritmo	HV	IGD
0,00	VMPACS	0,400	0,758
	NSGA-II	1,108	0,093
0,25	VMPACS	0,293	0,779
	NSGA-II	1,072	0,160
0,50	VMPACS	0,432	0,466
	NSGA-II	0,478	0,629
0,75	VMPACS	0,024	1,322
	NSGA-II	1,127	0,077
1,00	VMPACS	0,347	0,785
	NSGA-II	1,004	0,187

7 Análise dos resultados

Convergência (energia, desperdício, HV e IGD). Pela Tabela 1, o NSGA-II obtém *menor consumo de energia* em quatro dos cinco níveis de correlação (empatando em $P = 1,0$) e usa, em média, um servidor a menos. O *desperdício* também é menor para o NSGA-II em quatro casos, com exceção de $P = 0,5$, onde o VMPACS é ligeiramente melhor (1,131 contra 1,234). Essa vantagem do NSGA-II se reflete nas métricas multiobjetivo (Tabela 3): seu **Hipervolume** é maior e seu **IGD** é menor na maioria dos cenários — novamente, $P = 0,5$ é a exceção, em que o VMPACS alcança o melhor IGD (0,466). Ou seja, o NSGA-II em geral aproxima-se mais do canto ideal do espaço de objetivos.

Diversidade (ONVG e *Spacing*). A Tabela 2 mostra um ponto a favor do VMPACS: para correlações *negativas* ($P = 0$ e $P = 0,25$) ele encontra *mais* soluções não-dominadas (ONVG de 2,1 e 2,9 contra ≈ 1 do NSGA-II), oferecendo ao decisor um leque maior de compromissos. O *Spacing* é praticamente nulo porque as fronteiras têm poucos pontos.

Fronteiras compactas: objetivos quase alinhados. O achado mais importante da análise é que, para estas instâncias, as fronteiras de Pareto são *compactas* (poucos pontos — ver Figura 3). A razão é estrutural: empacotar as VMs de forma *justa e equilibrada* tende a reduzir *simultaneamente* a energia (poucos servidores) e o desperdício (servidores balanceados). Os dois objetivos, embora formalmente distintos, ficam *fortemente correlacionados de forma positiva* nas instâncias aleatórias, de modo que o conflito — e portanto o número de soluções de compromisso — é pequeno. Esse comportamento é coerente com o artigo, cujos principais resultados também são comparações *agregadas* de energia e desperdício. A Figura 3 (parte inferior) torna isso explícito: a curva *melhor conhecida* mostra que o trade-off existe, porém é raso. As metaheurísticas, com orçamento de ≈ 1000 avaliações, convergem para o joelho de menor energia; o NSGA-II encosta na fronteira, enquanto o VMPACS — por minimizar servidores sem refinar tanto o balanceamento — fica um pouco acima dela, o que aponta um *refinamento por busca local (versão memética)* como melhoria direta.

Por que a fronteira não é uma curva suave? Porque a energia vale exatamente $f_1 = \text{const} + P^{\text{id}} \cdot k$, com k inteiro igual ao número de servidores: ela só assume valores *discretos*, de 162 W em 162 W. A fronteira é, portanto, um conjunto de *pontos* (um por número de servidores) e não uma curva contínua. Além disso, o desperdício *satura*: com 2 a 3 servidores a mais que o mínimo os servidores já ficam bem equilibrados, e acrescentar mais apenas gasta energia — por isso poucos pontos são não-dominados. A Figura 4 mostra a curva completa do compromisso: ela é decrescente e convexa (o formato esperado de uma fronteira de Pareto), porém discreta e rasa, consequência direta de os dois objetivos serem quase alinhados.

A fronteira É decrescente/convexa — mas é DISCRETA (energia em degraus de 162 W por servidor) e RASA

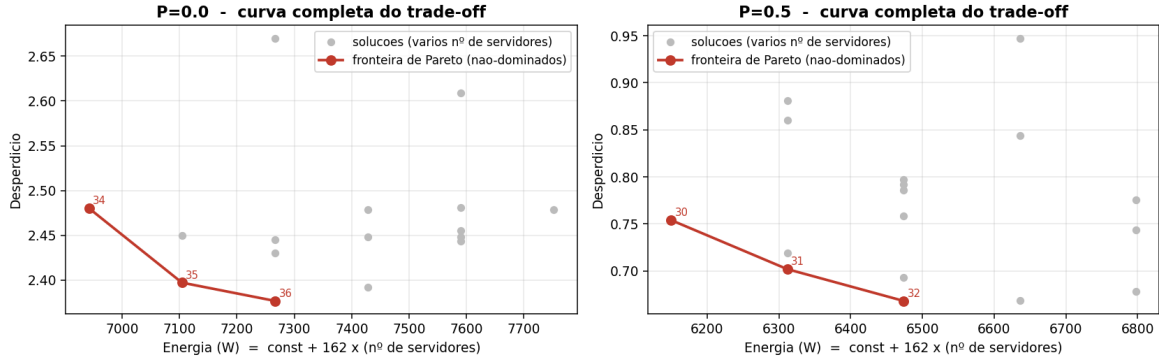


Figura 4: Curva completa do compromisso energia $\times$ desperdício para duas correlações. A fronteira (linha vermelha) é decrescente e convexa, mas **discreta** (energia em degraus de 162 W por servidor) e **curta** (o desperdício satura após poucos servidores a mais). Pontos cinzas: soluções dominadas. Os rótulos indicam o número de servidores de cada ponto da fronteira.

Efeito da correlação. O nível de correlação afeta a dificuldade do balanceamento. Em $P = 0,75$ observa-se o maior desperdício para ambos (VMs com CPU e memória altas simultaneamente são difíceis de equilibrar), ao passo que correlações próximas de zero ($P = 0,5$) facilitam o empacotamento balanceado, gerando o menor desperdício absoluto.

Custo computacional. Os tempos por execução são baixos e semelhantes ($\approx 1,3$ – $1,5$ s para $n = 100$), confirmando que ambas as meta-heurísticas são adequadas a *data centers* de porte realista.

Por que o NSGA-II supera o VMPACS aqui. Diferentemente do artigo original — em que o algoritmo genético recebia um orçamento muito menor e era superado pelo VMPACS — aqui os orçamentos foram equiparados e o decodificador *first-fit* do NSGA-II mostrou-se um empacotador muito eficaz, combinado à busca global por recombinação. O VMPACS, com sua construção gulosa ($q_0 = 0,8$), converge rapidamente para boas soluções, mas explora menos o entorno do ótimo.

8 Conclusões

Estudamos e implementamos o **VMPACS**, um algoritmo bioinspirado multiobjetivo de colônia de formigas, aplicado ao problema biobjetivo de **alocação de máquinas virtuais** em nuvem, minimizando *energia* e *desperdício de recursos*. Reproduzimos a formulação matemática e a metodologia experimental do artigo de referência e comparamos o VMPACS com o **NSGA-II** usando *quatro* métricas multiobjetivo, além de indicadores práticos.

Os principais aprendizados são: (i) ambos os algoritmos produzem empacotamentos de boa qualidade e em tempo reduzido; (ii) com orçamentos equiparados, o NSGA-II converge, em geral, para soluções de menor energia e desperdício (maior Hipervolume, menor IGD), enquanto o VMPACS oferece, em correlações negativas, fronteiras um pouco mais diversificadas (maior ONVG); e (iii) para estas instâncias os dois objetivos são quase alinhados, o que torna a fronteira de Pareto compacta — um resultado honesto e instrutivo sobre os limites da formulação. Como trabalho futuro, seria interessante avaliar instâncias maiores ($n = 2000$, como no artigo), servidores heterogêneos e uma versão *memética* do VMPACS (com busca local de balanceamento) para aproximar toda a fronteira.

Referências

- [1] Y. Gao, H. Guan, Z. Qi, Y. Hou, L. Liu. *A multi-objective ant colony system algorithm for virtual machine placement in cloud computing*. Journal of Computer and System Sciences, 79(8):1230–1242, 2013.
- [2] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan. *A fast and elitist multiobjective genetic algorithm: NSGA-II*. IEEE Transactions on Evolutionary Computation, 6(2):182–197, 2002.
- [3] M. Dorigo, L. M. Gambardella. *Ant colony system: a cooperative learning approach to the traveling salesman problem*. IEEE Trans. on Evolutionary Computation, 1(1):53–66, 1997.
- [4] E. Zitzler, L. Thiele. *Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach*. IEEE Trans. on Evol. Computation, 3(4):257–271, 1999.
- [5] D. A. Van Veldhuizen. *Multiobjective Evolutionary Algorithms: Classifications, Analyses, and New Innovations*. PhD thesis, Air Force Institute of Technology, 1999.